Required Capstone Checkpoint 5.1: Designing and Evaluating an Agent-Based RAG System

Directions

Download this worksheet and save a copy to your device before you begin working. Use this worksheet to plan, organize, and complete your submission for Required Capstone Checkpoint 5.1. Type your responses directly into the document, save your completed work, and upload the finished file when you are ready to submit.

Completion Checklist

☒ Description of the agent-based system and task
☒ Explanation of how the system coordinates retrieval, reasoning, and multistep execution
☒ Evaluation results from 2-3 representative multistep test cases
☒ Description of refinements made to the system
☒ Analysis of the impact of those refinements
☒ Reflection on system strengths, limitations, and future improvements

Step 1. System Design

Describe the task that your agent-based system is designed to complete.

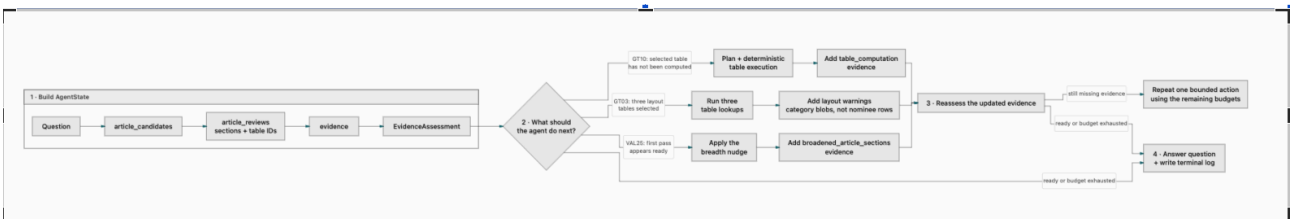
Examples may include generating reports, answering complex questions, synthesizing information across multiple documents, or completing another multistep task relevant to your capstone scenario.

Task description
I built an agent-based retrieval-augmented generation system that answers complex questions over an offline Wikipedia corpus. The task is not limited to finding a relevant page. The system must decide whether its evidence is sufficient, identify what is missing, choose the next retrieval or table operation, and produce an answer from a bounded and auditable evidence set. The benchmark includes questions that require several articles, exact article sections, or structured operations over tables. My design goal was to retain the strong source recall of earlier checkpoints while avoiding the cost and noise of sending full articles to the answer model. I also wanted the graph to make its decisions visible so that a failure could be traced to retrieval, section selection, table representation, planning, or final synthesis.

© MIT xPRO® 2026. All rights reserved.

Agent workflow: Describe how your system works

1. My agent starts by breaking the question into the main people, places, events, or topics named in it. It searches the Wikipedia section index for each part of the question. From those search results, it keeps up to four different Wikipedia articles that are most likely to contain the answer.
2. The agent then checks the title and table of contents of each article. This lets it judge the whole article instead of making a decision from one short passage. It removes articles that do not fit the question. For every article it keeps, it records the sections and tables that appear useful.
3. The graph reads the selected sections and tables and saves them in AgentState as evidence. AgentState is the shared record that moves from one step of the graph to the next. The evidence assessor reviews this record and chooses one of three results: there is enough information to answer, a table needs to be processed, or more information must be retrieved.
4. If the question needs information from a selected table, the graph sends that table to the table tool. This is a mandatory step for tables. The model describes the operation that is needed. For example, it may ask to find matching rows, filter rows by a value, count results, compare values, sort rows, or follow events over time. Python performs the operation. The result also records the original table cells that were used, so I can trace the answer back to its source.
5. If the question does not depend on a table, the graph may add a few more sections from the articles it already selected. If those sections still do not provide enough information, the graph uses links between Wikipedia articles to search related pages. The graph limits the number of retrieval and table attempts, so it cannot continue repeating the same work forever.
6. After each additional action, the assessor checks all of the collected evidence again. When the evidence is sufficient, or when the attempt limits have been reached, the graph stops retrieving and saves the final AgentState in a separate log for that question. The course test program then uses the evidence to generate an answer and grades it as pass, partial, or fail using the same rubric as the baseline systems.



Step 2. Initial Evaluation

Develop and evaluate your system using 2-3 representative test cases for your capstone scenario. Each test case should require more than one retrieval step or action.

For each test case, compare the agent-based system with your fixed retrieval pipeline from Checkpoint 2.1 or 3.1. Record relevant differences in answer quality, completeness, retrieval behavior, model calls, latency, or cost.

© MIT xPRO® 2026. All rights reserved.

Evaluation results

GT10 Winter Olympic program filter. The fixed Checkpoint 3.1 vector pipeline retrieved the correct article with recall 1.00 but failed all five recorded runs because the model misread the Sport and Years table.

The agent selected the exact table, recognized that its visible excerpt was incomplete, planned a year_min_gte(1950) filter, and returned the six required sport-year pairs with source-cell provenance. The annotated agent run passed.

GT03 Academy Awards intersection. The fixed vector pipeline found only one of the three required ceremony articles in each recorded run (recall 0.33) and failed.

The agent found all three, selected all three Winners and nominees tables, and executed three lookups (recall 1.00). It still failed because each normalized cell contained an entire award category rather than one nominee per row. This case localized the remaining error to table representation instead of retrieval or routing.

VAL25 stratovolcano explanation. This non-table trace shows the breadth route. (This part was able after validation set showed poor performance over plain text as I am no longer sending full articles)

The first assessment judged the selected sections sufficient, but the readiness policy required one same-article breadth round. Two additional sections were added, the second assessment became ready, and the answer passed. The fixed and agent variants also passed this case, so I use it to verify the control path rather than claim an individual score gain.

Across the table-heavy 19-question GT set, the Checkpoint 3.1 fixed vector baseline averaged 0.747 answer quality, 72.6% strict PASS, 0.899 article recall, and about 127K context characters per question. The forced table-gateway agent reached 0.789 quality, 75.0% PASS, 1.000 recall, and 44.2K context characters.

The separate 25-question validation set produced a more cautious result: the broadened agent reached 0.880 quality and 83.0% PASS versus 0.907 and 89.3% for the fixed 3.1 baseline, while using 48.3K rather than 130.9K context characters and improving recall from 0.973 to 1.000.

Strengths observed

- The final agent configurations maintained 1.000 designated-article recall on both the GT and validation evaluations.
- TOC review and exact section loading kept the evidence smaller and more focused than the earlier full-article pipelines.
- The deterministic table path improved table-dependent answers and retained exact source-cell provenance.
- Per-question AgentState logs made routing and failure analysis auditable instead of relying only on aggregate scores.

Weaknesses or challenges identified

- Selecting only a few sections can omit context needed for long prose or relationship synthesis even when the correct article was retrieved.
- Layout tables such as Academy Awards nominee grids do not yet normalize into the row structure required for deterministic intersections.
- The planner can choose the wrong temporal or counting operation, and forced table evidence can be unhelpful for uncertainty or source-conflict questions.
- The additional model calls improve control and diagnosis but add latency and nondeterminism.

© MIT xPRO® 2026. All rights reserved.

Step 3. System Refinement

Based on your evaluation results, make one or more improvements to your system. Examples may include:

- Improving system prompts
- Adjusting workflow structure
- Modifying retrieval behavior
- Improving how the agent uses retrieved information

Changes implemented
• Replaced a single blunt initial query with entity-aware decomposition and raised the article seed limit to four for questions with several named sources. • Replaced snippet-based article review with one TOC-based review that jointly selects articles, prose sections, and table artifact IDs. • Added run-once table normalization for spans, multi-row headers, formatting, superscripts, subscripts, legends, and stable source coordinates. • Added the table gateway so every selected table receives deterministic lookup or computation rather than depending on whether the assessor requests the tool. [tool became mandatory to combat assessor's mistakes and I introduced simple look up to table compute options] • Added a bounded same-article breadth route and first-pass readiness nudge for non-table synthesis questions, followed by graph-seeded retrieval only if needed. [this is to try bring answer score for text-only answer] • Added one terminal AgentState evidence log per question for review and debugging.
Justification
Each refinement came from a specific failure in the evidence logs. Entity queries were added when a full-question search buried a named article. TOC review was added when the correct article appeared in the candidate set but was rejected because its representative chunk was a lead or See also section. Normalized tables were added after flattened rows misaligned values and lost winner or annotation semantics. The gateway was added when GT10 passed only in runs where the assessor happened to request computation. Finally, validation showed that correct article recall did not guarantee enough prose context, which motivated one bounded same-article breadth round.

© MIT xPRO® 2026. All rights reserved.

Step 4. Impact of Refinement

Evaluate the updated system. Describe any changes in task completion, accuracy, relevance, or overall system behavior. Compare the refined agent-based system with both its initial version and the fixed baseline. Explain whether the refinement improved performance and identify any trade-offs.

Impact on system performance
On the table-heavy GT set, TOC-selected sections alone scored 0.579 quality and 40.8% strict PASS. Adding aligned normalized tables raised quality to 0.697 and PASS to 57.9%. Assessor-gated computation did not improve mean quality because the assessor sometimes skipped the tool. Requiring every selected table to pass through the gateway raised quality to 0.789 and PASS to 75.0%, with recall at 1.000. The held-out validation set exposed the trade-off. The section-focused agent scored 0.845 quality at 1.000 recall. One bounded breadth round raised quality to 0.880 while recall stayed at 1.000, but context grew from 35.5K to 48.3K characters. The result remained slightly below the 3.1 fixed baseline's 0.907 quality, although the agent used about 63% less context and had higher recall.
Comparison of initial and refined systems
The initial 5.1 system was effective at finding articles but weak at controlling what happened after retrieval. It could reject a correct article from one poor snippet, send incomplete table excerpts, or answer before a useful tool ran. The refined system makes those decisions explicit in AgentState and adds bounded routes for table computation, same-article breadth, and related-article retrieval. I was impressed with 5.1 agent holding consistent 1.0 article recall score. Compared with the fixed 3.1 pipeline, the refined agent improved recall and greatly reduced context. It also improved aggregate quality on the table-heavy GT set. It did not beat the fixed baseline on the broader validation set because selected sections sometimes removed prose needed for synthesis. I therefore view the agent as a better-controlled and more diagnosable pipeline, but not yet a universal replacement for broad-context retrieval.

© MIT xPRO® 2026. All rights reserved.

Step 5. Reflection

Reflect on your experience designing and evaluating an agent-based RAG system.

What contributed most to successful task completion?

The largest contribution was separating retrieval from evidence use.

Article recall told me whether the source was found while AgentState showed whether the article was kept, which sections and tables were loaded, which plan was executed, and why the graph stopped. This prevented me from treating every bad answer as another retrieval problem.

Retrieval validation loop allowed to try out different retrieval tools. Each retriever turn was preset to use different retriever capabilities to enhance previously kept data. This was very affective hold recall at 1.0 for both test GT and validation GT.

```
# Retriever strategy per stage. "multistep_entity" decomposes the question into one
# clean full-name query per named/implied entity at the initial stage, so entities a
# blunt query buries (e.g. GT16's Barack Obama) are retrieved at rank 1; graph
# expansion is the repeated escalation. Set INITIAL_RETRIEVER = "vector" to reproduce
# the plain-vector baseline for A/B, or "multistep" for sub-topic decomposition.
INITIAL_RETRIEVER = "multistep_entity"  # "vector" | "multistep" | "multistep_entity"
# Escalation stages after the initial one. article_broaden first — recall is already
# 1.000, so the first escalation's job is context breadth for synthesis questions the
# single-section bet starves (validation §3 Step 8-10); graph_seeded stays as the
# fallback for questions that genuinely need another article.
REPEATED_RETRIEVERS = ("article_broaden", "graph_seeded")
```

The deterministic table gateway was the strongest functional improvement because it turned a variable model judgment into a repeatable operation with auditable provenance.

What limitations remain?

The main unresolved issue is evidence granularity. A small section set is efficient but can starve broad explanations; restoring whole articles would improve coverage but would recreate the cost and noise I was trying to avoid. Some Wikipedia tables also encode meaning through layout rather than rows, so deterministic execution cannot repair them after parsing. Planner mistakes remain possible, multi-table results are still joined by the answer model, and evidence assessment is an LLM judgment that requires repeated runs to evaluate reliably.

What improvements would you prioritize next?

Full article vs sections.
This is the issue that I don’t think was covered very well yet. How do I ensure I have enough information, considering that sending a full article is not always an option due to context constraint and “lost in the middle”

© MIT xPRO 2026. All rights reserved.

effect. I suspect it would be even more of an issue with large research papers. Simple retrieval is not enough. Data processing for RAG is an important lesson that is missing.

If I had time, I would try to improve section look up and understand specific nuances that make my section selection approach incomplete. I would try a strong model for assess and judge portion to see if incomplete data can be better identified. And in the end because wiki pages are still fairly controlled in size I would explore fallback on a full article (it would not be an option for research papers)

2. More table work based on GT03. Detect and explode layout tables into nominee-level or record-level rows before computation, beginning with the Academy Awards failure in GT03. The tricky part is understand the current shortcoming as a generic implementation and not to overfit it for just Academy Awards specific formatting. I would also add stronger validation for temporal aggregates and deterministic joins across multiple table results before changing the model itself.

Supporting Materials

Use this section to list, describe, or link to any supporting materials included with your submission.

Supporting material	Location, link, or note
Retriever Evalution HTML	retriever_phylogeny.html
Final Evaluation results	ragas_agent_toc_and_tables_v6.pdf and ragas_agentic_retriever_5.1_validation_set_v2.pdf.
Deterministic Table Computation	README_ComputeTable.md
Other supporting evidence	Annotated excerpts from AgentState logs for GT03, GT10, and VAL25 AgentState samples. (all logs are preserved in the repo)

© MIT xPRO® 2026. All rights reserved.

AI Assistance Disclosure

I used AI to assist with organizing my code and building different utilities to understand the current evolution. AI also helped me with breaking down tables into json format that was instrumental for deterministic table compute

In addition, I used AI to organize my observations and refine the wording of my written analysis.

The performance evaluation and narrowing down on next steps for improvement were done by me, including the agentic graph design.

I reviewed the prompts, experimental runs, interpretations, and final conclusions and made all final decisions regarding final outcomes and evidence to be taken into account for this report.

End of Required Capstone Checkpoint 5.1

Return to the assignment page in the learning platform to upload and submit your completed capstone checkpoint worksheet and complete any remaining items in this module.

© MIT xPRO® 2026. All rights reserved.